

NETWORKWALKS

Penetration Testing Report

Mediroza General Hospital

Client: Mediroza General Hospital

Target: <https://medirozahospital.com>

Engagement Type: Black-box Penetration Test

Prepared by: Mikias Mekite - Networkwalks Batch B083

Authorization: Written permission granted by the client for this engagement.

CONFIDENTIAL: This report contains sensitive security findings and is intended solely for the client and authorized personnel.

1. Executive Summary

I conducted a black-box penetration test of Mediroza General Hospital's public web application, hosted at <https://medirozahospital.com>. The objective was to identify exploitable vulnerabilities, demonstrate their real-world impact, and provide actionable remediation guidance.

Testing uncovered multiple critical weaknesses that, when chained together, allowed a fully unauthenticated attacker to bypass patient portal authentication, access confidential patient lab reports, and retrieve a complete internal database backup containing staff personal data, salaries, and shareholder ownership records. No credentials, insider access, or social engineering were required to obtain any of this data.

The overall risk to Mediroza General Hospital from these findings is assessed as CRITICAL. The vulnerabilities directly expose regulated patient health information (PHI), employee personal and financial data, and confidential corporate ownership information, creating significant legal, regulatory, financial, and reputational risk.

2. Scope and Methodology

2.1 Scope

- Target: <https://medirozahospital.com>
- Engagement type: Full black-box penetration test
- Testing limited strictly to the target domain
- Out of scope: social engineering, denial-of-service testing, any testing outside the agreed domain

2.2 Methodology

Testing followed a standard black-box methodology, proceeding from reconnaissance through exploitation:

- Passive and active reconnaissance of the target domain
- Enumeration of publicly accessible files and directories (robots.txt, sitemap.xml)
- Manual review of application entry points, including forms and file-serving scripts
- Authentication mechanism testing, including input handling and error-message analysis
- Manual SQL injection testing against login functionality
- Directory and file enumeration of paths disclosed via robots.txt
- Offline password recovery against protected documents
- Metadata analysis of retrieved documents

2.3 Tools Used

- Web browser and built-in developer tools (source view, network inspection)
- Manual SQL injection payloads
- Networkwalks Hash Calculator and Password Cracker (with expanded custom wordlist)
- PDF metadata inspection

2.4 Limitations

This was a black-box assessment with no credentials or internal documentation provided at the outset. Testing was limited to the in-scope domain; no infrastructure, network, or physical testing was performed.

3. Findings and Proof of Exploitation

F1, SQL Injection: Authentication Bypass (Critical)

Location: Patient Portal login (medirozahospital.com/patient/login.php)

Description: The patient portal login form was found to construct its database query by directly concatenating user-supplied input, without sanitization or parameterization. This allowed authentication to be bypassed entirely using a classic SQL injection payload.

Proof of Exploitation:

- Payload used in the Username field: admin' --
- Any value submitted in the Password field
- Result: the application logged the tester in without a valid password, granting access to the patient portal and its file-download functionality

Impact: An unauthenticated attacker can gain full access to the patient portal and any confidential documents it serves, without needing valid credentials. This directly enabled retrieval of three confidential patient lab reports (see deliverable evidence in the project repository).

Root Cause: The login query appended raw user input directly into a SQL statement. The injected quote character closed the intended string early, and the trailing comment marker (--) caused the database to ignore the remainder of the query, including the password check.

F2, Sensitive Database Backup Publicly Exposed (Critical)

Location: medirozahospital.com/old/mediroza_db_backup_2019.sql

Description: A full SQL database backup file was found publicly accessible under the /old/ directory, discoverable after reviewing the site's robots.txt file, which disallowed but did not actually block the path.

Proof of Exploitation:

- File directly downloadable with no authentication: /old/mediroza_db_backup_2019.sql
- Contents included a complete 'staff' table: full names, job titles, departments, emails, phone numbers, national ID numbers, and monthly salaries for 30 employees
- Contents also included a complete 'shareholders' table: shareholder names, share percentages, shares held, and share class

Impact: This single file exposes highly sensitive personal data (including national identification numbers), payroll information for every employee, and confidential corporate ownership records. Exposure of national ID numbers combined with contact details creates a significant identity theft risk for staff. Exposure of shareholder data represents a serious breach of corporate confidentiality.

Root Cause: An outdated backup file was left in a publicly reachable web directory rather than being stored outside the webroot or removed after use.

F3, Directory Listing Enabled on Sensitive Paths (Medium)

Location: /old/, /patient/, /staff/

Description: The web server returns a full directory listing ("Index of ...") when these folders are accessed directly, revealing every file within them instead of returning a 403 or 404 response.

Proof of Exploitation:

- /old/ listed mediroza_db_backup_2019.sql directly
- /patient/ listed download.php, error_log, login.php, logout.php, portal.php, and a reports/ folder
- /staff/ listed login.php

Impact: Directory listing significantly lowers the effort required for an attacker to discover sensitive files, as seen directly in Finding F2. Even where individual files are protected, listing exposes the application's internal structure and naming conventions, aiding further attacks.

Root Cause: Directory browsing/indexing was not disabled at the web server configuration level for these directories.

F4, Sensitive Paths Disclosed via robots.txt (Low)

Location: /robots.txt

Description: The site's robots.txt file explicitly lists /patient/, /staff/, and /old/ as disallowed paths. While intended to prevent search engine indexing, this file is publicly readable and effectively provided a roadmap of sensitive areas to investigate.

Proof of Exploitation:

- robots.txt content: Disallow: /patient/, Disallow: /staff/, Disallow: /old/

Impact: On its own this is low risk, since robots.txt is a standard, expected file. However, combined with F3 (directory listing enabled) it directly led to the discovery of Finding F2, escalating its practical severity.

Root Cause: Reliance on robots.txt as a means of hiding sensitive directories, rather than enforcing actual access control on those paths.

F5, Document Metadata Leakage (Medium)

Location: Retrieved patient PDF lab reports

Description: Two of the three retrieved patient lab report PDFs listed "Mediroza Hospital" as the document author, consistent with normal system-generated reports. The third PDF listed the author as "jmalik" matching Jameel Malik, the hospital's IT Systems Administrator, as identified in the exposed staff database (Finding F2).

Proof of Exploitation:

- PDF metadata (Author field) inspected for all three retrieved reports
- Third report's author field matched the IT Systems Administrator's account name rather than the hospital's standard reporting system

Impact: This strongly suggests that a patient lab report was generated, edited, or handled directly by IT staff outside of the normal clinical reporting process, indicating a possible internal access-control gap around who can view or produce confidential patient records. This warrants further internal investigation by the hospital.

Root Cause: Insufficient separation between IT/administrative system access and confidential patient health record handling.

4. Risk Rating

Each finding was rated using a standard four-tier scale (Critical, High, Medium, Low), based on the sensitivity of the data or access exposed, the ease of exploitation, and the number of steps required.

Rating Justification

- F1 (Critical): Requires no credentials, no special access, and no prior knowledge; a single crafted input yields full authentication bypass and direct access to confidential patient data.
- F2 (Critical): A single unauthenticated request exposes complete, unredacted PII, national ID numbers, salary data, and shareholder records for the entire organization.

- F3 (Medium): Not independently exploitable but materially increases the impact and discoverability of other findings.
- F4 (Low): Standard, expected file; only becomes meaningful in combination with F3.
- F5 (Medium): Does not by itself grant technical access, but indicates a possible internal process/access-control failure with data-protection implications.

5. Recommendations and Remediation

F1, SQL Injection

- Rewrite all database queries to use parameterized queries / prepared statements instead of string concatenation.
- Apply strict server-side input validation on all authentication fields.
- Conduct a full code review of all other forms and scripts for the same vulnerability pattern.

F2, Exposed Database Backup

- Immediately remove the exposed backup file from the public web directory.
- Store all backups outside the webroot, in a location not reachable via HTTP.
- Encrypt backup files at rest and restrict access to authorized administrators only.
- Establish a retention and secure-deletion policy for old backups.

F3, Directory Listing

- Disable directory listing/indexing at the web server configuration level for all directories.
- Add explicit index files or return 403 Forbidden for directories with no intended public listing.

F4, robots.txt Disclosure

- Do not rely on robots.txt to hide sensitive paths; enforce authentication and authorization at the application/server level instead.
- Consider removing explicit sensitive path names from robots.txt where server-side access control already exists.

F5, Metadata / Access Control

- Review and restrict which staff roles and accounts are able to access or generate patient lab reports.
- Enable audit logging for all access to patient records, including by IT/administrative accounts.
- Strip or standardize document metadata before reports are delivered to patients.

General Recommendations

- Conduct regular penetration testing and vulnerability scanning as part of an ongoing security program.
- Implement a Web Application Firewall (WAF) as a defense-in-depth measure.
- Provide secure coding training to the development team, with a focus on injection vulnerabilities and safe file handling.

ID	Finding	Risk
F1	SQL Injection - Authentication Bypass	Critical
F2	Sensitive Database Backup Publicly Exposed	Critical
F3	Directory Listing Enabled on Sensitive Paths	Medium
F4	Sensitive Paths Disclosed via robots.txt	Low
F5	Document Metadata Leakage (Improper Access Indicator)	Medium

ID	Finding	Risk
F1	SQL Injection - Authentication Bypass	Critical
F2	Sensitive Database Backup Publicly Exposed	Critical
F3	Directory Listing Enabled on Sensitive Paths	Medium
F4	Sensitive Paths Disclosed via robots.txt	Low
F5	Document Metadata Leakage (Improper Access Indicator)	Medium